Bubble Sort and Insertion Sort

Data Structures Using C – Sorting Algorithms

Department of Computer Science and Engineering

Graphic Era (Deemed to be University), Dehradun

Recall: What is Sorting?

Sorting means arranging data in a particular order.

Ascending Order

$2,\ 5,\ 9,\ 13,\ 20$

Smallest to largest.

Descending Order

$20,\ 13,\ 9,\ 5,\ 2$

Largest to smallest.

Consider:

$$A = [30,\ 52,\ 29,\ 87,\ 63,\ 27,\ 19,\ 54]$$

Our goal is to arrange these values in ascending order.

Two Simple Sorting Approaches

Bubble Sort

Compare neighbouring elements.
Swap them when they are in the wrong order.
After each pass, one large element reaches its final position.

Insertion Sort

Maintain a sorted portion.
Pick the next element from the unsorted portion.
Insert it at its correct position in the sorted portion.

Bubble Sort: Compare and Swap

Insertion Sort: Pick, Shift and Insert

Bubble Sort: Introduction

Bubble Sort is a simple comparison-based sorting algorithm.

- It repeatedly compares adjacent elements.
- If the left element is greater than the right element, the two elements are swapped.
- After the first pass, the largest element reaches the last position.
- After the second pass, the second-largest element reaches its correct position.
- This process continues until the array is sorted.

Important Observation

In ascending Bubble Sort, the largest remaining element moves towards the end of the unsorted portion.

Bubble Sort: Basic Idea

Consider:

$$A = [5,\ 2,\ 4,\ 1]$$

Initial		5	2	4	1
Compare 5 and 2	$\Rightarrow$	2	5	4	1
Compare 5 and 4	$\Rightarrow$	2	4	5	1
Compare 5 and 1	$\Rightarrow$	2	4	1	5

At the end of Pass 1:

5 is in its final position

Bubble Sort: Pass-by-Pass Example

Consider the array:

$$A = [5,\ 2,\ 4,\ 1]$$

Stage	Array			
Initial	5	2	4	1
Pass 1	2	4	1	5
Pass 2	2	1	4	5
Pass 3	1	2	4	5

Total passes required in the standard algorithm:

$$n - 1 = 4 - 1 = 3$$

Bubble Sort: Comparisons per Pass

For an array of size n:

Pass	Comparisons	Element Fixed
1	$n-1$	Largest
2	$n-2$	Second largest
3	$n-3$	Third largest
$\vdots$	$\vdots$	$\vdots$
$n-1$	1	Remaining order

Total comparisons:

$$(n-1)+(n-2)+\cdots+2+1$$

$$=\frac{n(n-1)}{2}$$

Textbook Example: Bubble Sort

Consider the array:

$$A = [30,\ 52,\ 29,\ 87,\ 63,\ 27,\ 19,\ 54]$$



- Number of elements: $n = 8$
- Maximum standard passes: $n - 1 = 7$
- First comparison: 30 and 52
- Since $30 < 52$, no swapping is required.

Bubble Sort: First Pass – Part 1

Initial array:

$$[30,\ 52,\ 29,\ 87,\ 63,\ 27,\ 19,\ 54]$$

1. Compare 30 and 52: no swap.
2. Compare 52 and 29: swap.
3. Compare 52 and 87: no swap.
4. Compare 87 and 63: swap.

Array after these comparisons:

$$[30,\ 29,\ 52,\ 63,\ 87,\ 27,\ 19,\ 54]$$

Bubble Sort: First Pass – Part 2

Continue from:

$$[30,\ 29,\ 52,\ 63,\ 87,\ 27,\ 19,\ 54]$$

1. Compare 87 and 27: swap.
2. Compare 87 and 19: swap.
3. Compare 87 and 54: swap.

After Pass 1:

$$[30,\ 29,\ 52,\ 63,\ 27,\ 19,\ 54,\ 87]$$

Observation

The largest value, 87, is now at index 7. It will not participate in the next pass.

Bubble Sort: Remaining Passes

Pass	Array After Pass
Initial	30, 52, 29, 87, 63, 27, 19, 54
1	30, 29, 52, 63, 27, 19, 54, 87
2	29, 30, 52, 27, 19, 54, 63, 87
3	29, 30, 27, 19, 52, 54, 63, 87
4	29, 27, 19, 30, 52, 54, 63, 87
5	27, 19, 29, 30, 52, 54, 63, 87
6	19, 27, 29, 30, 52, 54, 63, 87
7	19, 27, 29, 30, 52, 54, 63, 87

19, 27, 29, 30, 52, 54, 63, 87

Bubble Sort: Algorithm

BUBBLE_SORT(A, N)

1. Repeat for $i = 0$ to $N - 2$.
2. Repeat for $j = 0$ to $N - i - 2$.
3. Compare $A[j]$ and $A[j + 1]$.
4. If $A[j] > A[j + 1]$, swap them.
5. End inner loop.
6. End outer loop.

Number of comparisons in pass $i = N - i - 1$

Bubble Sort: C Function

```c
void bubbleSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

Key Point

The inner loop excludes the sorted suffix after each pass.

Bubble Sort: Complete C Program

```c
#include <stdio.h>

void bubbleSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1])
            {
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
}

int main(void)
{
    int a[] = {30, 52, 29, 87, 63, 27, 19, 54};
    int n = sizeof(a) / sizeof(a[0]);

    bubbleSort(a, n);

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
    return 0;
}
```

Bubble Sort: Complexity Analysis

For the standard implementation:

$$T(n) = (n-1) + (n-2) + \cdots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \Theta(n^2)$$

Case	Time Complexity
Best	$\Theta(n^2)$
Average	$\Theta(n^2)$
Worst	$\Theta(n^2)$
Auxiliary space	$O(1)$

Why Optimize Bubble Sort?

Consider an already sorted array:

$$[10,\ 20,\ 30,\ 40,\ 50]$$

No swaps are necessary.
However, the standard Bubble Sort still performs:

$$4 + 3 + 2 + 1 = 10$$

comparisons.

Optimization

If an entire pass finishes without any swap, the array is already sorted.
We can stop immediately.

Optimized Bubble Sort: C Function

```c
void optimizedBubbleSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int swapped = 0;

        for (int j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;

                swapped = 1;
            }
        }

        if (swapped == 0)
            break;
    }
}
```

Why Must the Swap Flag Be Reset?

Correct Logic

At the beginning of every pass:

$$swapped = 0$$

If any swap occurs:

$$swapped = 1$$

At the end of the pass:

$$swapped = 0 \Rightarrow \text{Array is sorted}$$

Common Programming Error

Do not initialize the swap flag only once before the outer loop.
It must be reset at the beginning of each pass.

Optimized Bubble Sort: Complexity

Case	Standard	Optimized
Best	$\Theta(n^2)$	$\Theta(n)$
Average	$\Theta(n^2)$	$\Theta(n^2)$
Worst	$\Theta(n^2)$	$\Theta(n^2)$
Space	$O(1)$	$O(1)$

- Best case: already sorted array.
- Worst case: reverse-sorted array.
- Optimized Bubble Sort can terminate early.
- Both implementations sort in place.
- Both are stable when swaps occur only for $>$.

Bubble Sort: Stability

Consider records:

$$[(4, A),\ (2, B),\ (4, C)]$$

Here, the number is the sort key.
The letters identify the original records.

After stable sorting:

$$[(2, B),\ (4, A),\ (4, C)]$$

Notice that:

$$(4, A)$$

still appears before:

$$(4, C)$$

Bubble Sort: Classroom Activity

Sort the array:

$$[7,\ 3,\ 9,\ 2,\ 5]$$

using Bubble Sort.

1. Show the array after Pass 1.
2. Show the array after Pass 2.
3. Identify the element fixed after each pass.
4. Count the comparisons in Pass 1.
5. Determine the total comparisons in standard Bubble Sort.

$$n = 5$$

$$\text{Total comparisons} = \frac{5(5-1)}{2} = 10$$

Bubble Sort Activity: Solution

Stage	Array
Initial	7, 3, 9, 2, 5
Pass 1	3, 7, 2, 5, 9
Pass 2	3, 2, 5, 7, 9
Pass 3	2, 3, 5, 7, 9
Pass 4	2, 3, 5, 7, 9

$$\boxed{2,\ 3,\ 5,\ 7,\ 9}$$

Insertion Sort: Introduction

Insertion Sort builds a sorted array one element at a time.

- The first element is initially considered sorted.
- The next element is selected as the key.
- Larger elements in the sorted portion are shifted right.
- The key is inserted at its correct position.
- The sorted portion grows after each iteration.

Real-Life Analogy

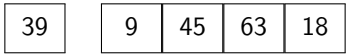
Arranging playing cards in your hand:
Pick a card and insert it at the correct position among the cards already arranged.

Insertion Sort: Two Partitions

Consider:

$$A = [39,\ 9,\ 45,\ 63,\ 18]$$

Initially:



Sorted Partition

Initially contains only the first element.

$[39]$

Unsorted Partition

Contains the remaining elements.

$[9,\ 45,\ 63,\ 18]$

Insertion Sort: First Pass

Initial array:

$$[39,\ 9,\ 45,\ 63,\ 18]$$

Select:

$$KEY = 9$$

Compare with 39:

$$39 > 9$$

Shift 39 one position to the right.
Insert 9 at index 0.

After Pass 1:

$$\boxed{[9,\ 39,\ 45,\ 63,\ 18]}$$

Sorted portion:

Insertion Sort: Second and Third Passes

Current array:

$$[9,\ 39,\ 45,\ 63,\ 18]$$

Pass 2

$$KEY = 45$$

Since:

$$39 < 45$$

no shifting is required.

Pass 3

$$KEY = 63$$

Since:

Insertion Sort: Fourth Pass

Current array:

$$[9,\ 39,\ 45,\ 63,\ 18]$$

Select:

$$KEY = 18$$

1. $63 > 18$: shift 63.
2. $45 > 18$: shift 45.
3. $39 > 18$: shift 39.
4. $9 < 18$: stop shifting.

Insert 18 after 9.

$$\boxed{[9,\ 18,\ 39,\ 45,\ 63]}$$

Insertion Sort: Pass-by-Pass Summary

Pass	Array After Pass	Key
Initial	39, 9, 45, 63, 18	–
1	9, 39, 45, 63, 18	9
2	9, 39, 45, 63, 18	45
3	9, 39, 45, 63, 18	63
4	9, 18, 39, 45, 63	18

After Pass i, the first $i + 1$ elements are sorted.

Insertion Sort: Algorithm

INSERTION_SORT(A, N)

1. Repeat for $i = 1$ to $N - 1$.
2. Set $KEY = A[i]$.
3. Set $j = i - 1$.
4. While $j \geq 0$ and $A[j] > KEY$:
   1. Set $A[j + 1] = A[j]$.
   2. Decrement j.
5. Set $A[j + 1] = KEY$.
6. Repeat for the next element.

Important

Always check $j \geq 0$ before accessing $A[j]$.

Insertion Sort: C Function

```c
void insertionSort(int a[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }
}
```

Key Observation

Insertion Sort shifts elements rather than repeatedly swapping the key with its predecessor.

Insertion Sort: Complete C Program

```c
#include <stdio.h>

void insertionSort(int a[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }
}

int main(void)
{
    int a[] = {500, 1, 50, 23, 76};
    int n = sizeof(a) / sizeof(a[0]);

    insertionSort(a, n);
```

Insertion Sort: Expected Output

Input:

```
1 500 1 50 23 76
```

Output:

```
1 1 23 50 76 500
```

Explanation

The array is sorted by inserting each new element into its correct position in the already sorted prefix.

Insertion Sort: Best Case

Consider an already sorted array:

$$[10,\ 20,\ 30,\ 40,\ 50]$$

Each key is compared only once with the previous element.
No shifting is required.

Number of key comparisons:

$$n-1$$

Therefore:

$$\boxed{T(n) = \Theta(n)}$$

Insertion Sort: Worst Case

Consider a reverse-sorted array:

$$[50,\ 40,\ 30,\ 20,\ 10]$$

Every new key must be inserted at the beginning of the sorted portion.

Number of shifts:

$$1 + 2 + 3 + \cdots + (n-1)$$

$$= \frac{n(n-1)}{2}$$

Therefore:

$$\boxed{T(n) = \Theta(n^2)}$$

Insertion Sort: Complexity Analysis

Property	Insertion Sort
Best-case time	$\Theta(n)$
Average-case time	$\Theta(n^2)$
Worst-case time	$\Theta(n^2)$
Auxiliary space	$O(1)$
Stable	Yes
In-place	Yes
Adaptive	Yes

Insertion Sort is particularly useful when the data is already nearly sorted.

Why is Insertion Sort Stable?

Consider:

$$[(5, A),\ (3, B),\ (5, C)]$$

After sorting:

$$[(3, B),\ (5, A),\ (5, C)]$$

The two records having key 5 preserve their original relative order.

Reason

Insertion Sort shifts elements only when:

$$A[j] > KEY$$

Equal elements are not shifted past one another.

Advantages of Insertion Sort

- Simple to understand and implement.
- Efficient for small arrays.
- Performs well for nearly sorted data.
- Requires only constant auxiliary space.
- Stable when implemented using strict comparison.
- Can be adapted to process elements as they arrive.

Limitation

For large randomly ordered arrays, its quadratic time complexity makes it less efficient than advanced sorting algorithms.

Insertion Sort: Classroom Activity

Sort the array:

$$[8,\ 3,\ 6,\ 2,\ 7]$$

using Insertion Sort.

For every pass:

1. Identify the key.
2. Show all necessary shifts.
3. Show the array after insertion.
4. Identify the sorted portion.

Additional Question:
How many shifts are required in total?

Insertion Sort Activity: Solution

Pass	Key	Array After Pass	Shifts
Initial	–	8, 3, 6, 2, 7	–
1	3	3, 8, 6, 2, 7	1
2	6	3, 6, 8, 2, 7	1
3	2	2, 3, 6, 8, 7	3
4	7	2, 3, 6, 7, 8	1

Total shifts:

$$1 + 1 + 3 + 1 = \boxed{6}$$

Bubble Sort vs Insertion Sort

Property	Bubble Sort	Insertion Sort
Main operation	Adjacent swaps	Shifts and insertion
Best case (optimized)	$\Theta(n)$	$\Theta(n)$
Average case	$\Theta(n^2)$	$\Theta(n^2)$
Worst case	$\Theta(n^2)$	$\Theta(n^2)$
Auxiliary space	$O(1)$	$O(1)$
Stable	Yes	Yes
In-place	Yes	Yes
Adaptive version	Optimized version	Standard version
Nearly sorted data	Good with optimization	Often preferable

When Should We Use Each Algorithm?

Bubble Sort

Useful for:

- Learning adjacent comparisons.
- Understanding swapping.
- Demonstrating loop optimization.
- Small educational examples.

Insertion Sort

Useful for:

- Small datasets.
- Nearly sorted arrays.
- Incrementally arriving data.
- Situations requiring a stable, in-place sorting algorithm.

MCQ 1: Bubble Sort

After the first complete pass of ascending Bubble Sort, which element is guaranteed to be in its final position?

1. Smallest element
2. Largest element
3. Middle element
4. First element

MCQ 1: Bubble Sort

After the first complete pass of ascending Bubble Sort, which element is guaranteed to be in its final position?

1. Smallest element
2. Largest element
3. Middle element
4. First element

Answer

Option 2: Largest element

MCQ 2: Bubble Sort

How many comparisons are performed by standard Bubble Sort on an array of size n?

1. n
2. $n - 1$
3. $\frac{n(n-1)}{2}$
4. $n \log n$

MCQ 2: Bubble Sort

How many comparisons are performed by standard Bubble Sort on an array of size n?

1. n
2. $n-1$
3. $\frac{n(n-1)}{2}$
4. $n \log n$

Answer

$$\boxed{\frac{n(n-1)}{2}}$$

MCQ 3: Insertion Sort

In Insertion Sort, the array is conceptually divided into:

1. Left and right halves
2. Sorted and unsorted portions
3. Even and odd positions
4. Minimum and maximum portions

MCQ 3: Insertion Sort

In Insertion Sort, the array is conceptually divided into:

1. Left and right halves
2. Sorted and unsorted portions
3. Even and odd positions
4. Minimum and maximum portions

Answer

Option 2: Sorted and unsorted portions

MCQ 4: Insertion Sort

Which condition correctly shifts elements during ascending Insertion Sort?

1. `a[j] < key`
2. `a[j] > key`
3. `a[j] == key`
4. `a[j] != key`

MCQ 4: Insertion Sort

Which condition correctly shifts elements during ascending Insertion Sort?

1. `a[j] < key`
2. `a[j] > key`
3. `a[j] == key`
4. `a[j] != key`

Answer

Option 2: `a[j] > key`

MCQ 5: Complexity

Which statement is correct?

1. Standard Bubble Sort has $O(\log n)$ worst-case time.
2. Insertion Sort requires $O(n)$ auxiliary space.
3. Optimized Bubble Sort has $O(n)$ best-case time.
4. Insertion Sort is always faster than Merge Sort.

MCQ 5: Complexity

Which statement is correct?

1. Standard Bubble Sort has $O(\log n)$ worst-case time.
2. Insertion Sort requires $O(n)$ auxiliary space.
3. Optimized Bubble Sort has $O(n)$ best-case time.
4. Insertion Sort is always faster than Merge Sort.

Answer

Option 3: Optimized Bubble Sort has $O(n)$ best-case time.

Common Programming Mistakes

1. Accessing `a[j+1]` outside array bounds.
2. Using the wrong inner-loop limit in Bubble Sort.
3. Forgetting to reset the swap flag.
4. Accessing `a[j]` before checking `j >= 0`.
5. Losing the insertion key during shifting.
6. Confusing swaps with shifts.
7. Assuming both algorithms take linear time in all cases.

Important

Correct array bounds are essential in C because out-of-bounds access causes undefined behaviour.

Practice Questions

1. Explain Bubble Sort with a suitable example.
2. Write the algorithm and C program for Bubble Sort.
3. Derive the number of comparisons in Bubble Sort.
4. Explain optimized Bubble Sort.
5. Explain Insertion Sort using sorted and unsorted portions.
6. Write a C program for Insertion Sort.
7. Compare the stability and space requirements of both algorithms.
8. Which algorithm would you prefer for a nearly sorted array? Justify.

Lecture Summary

Bubble Sort

- Compares and swaps adjacent elements.
- Largest element reaches the end after each pass.
- Standard time: $\Theta(n^2)$.
- Optimized best case: $\Theta(n)$.

Insertion Sort

- Builds a sorted prefix one element at a time.
- Uses shifting to insert each key.
- Best case: $\Theta(n)$.
- Average and worst case: $\Theta(n^2)$.

References

1. Reema Thareja, *Data Structures Using C*, chapter on Searching and Sorting.
2. Ellis Horowitz, Sartaj Sahni, *Fundamentals of Data Structures*.
3. Thomas H. Cormen et al., *Introduction to Algorithms*.

Lecture Feedback

Your Feedback Matters

Please share your feedback on:

- Sorting concepts
- Bubble Sort passes
- Optimized Bubble Sort
- Insertion Sort dry runs
- C implementations
- Complexity analysis



Scan to Submit Feedback

Thank You

Questions and Discussion

Bubble Sort: Compare and Swap

Insertion Sort: Shift and Insert